

MANUAL DE USUARIO – YOGURTBATCH API

1. ¿Qué es este sistema?

YogurtBatch API es un sistema que permite simular y gestionar la producción de yogurt por lotes, controlando recetas, procesos de fermentación y registros de temperatura.

Este sistema está diseñado para ser consumido mediante herramientas como Swagger o aplicaciones externas.

2. ¿Cómo acceder al sistema?

El sistema está desplegado en la nube y no requiere instalación local.

API en producción:

<https://yogurtbatch.onrender.com>

Documentación Swagger:

<https://yogurtbatch.onrender.com/swagger-ui/index.html>

Desde Swagger puedes:

- Ver endpoints
- Probar requests
- Ver respuestas en tiempo real

Swagger `/v3/api-docs` Explore

OpenAPI definition v0 OAS 3.0
[/v3/api-docs](#)

Servers
https://yogurtbatch.onrender.com - Generated server url

Recetas Recetas de yogurt

- GET `/api/recipes/{id}` Obtener receta por ID
- PUT `/api/recipes/{id}` Actualizar receta
- GET `/api/recipes` Listar recetas
- POST `/api/recipes` Crear receta
- PATCH `/api/recipes/{id}/deactivate` Desactivar receta
- PATCH `/api/recipes/{id}/activate` Activar receta
- GET `/api/recipes/search` Buscar recetas

yogurt-batch-controller

- GET `/api/batches`
- POST `/api/batches`

3. Uso del sistema paso a paso

Paso 1: Entrar a Swagger

Abrir el enlace de Swagger y visualizar los endpoints disponibles.

GET	/api/recipes/{id}	Obtener receta por ID	▼
PUT	/api/recipes/{id}	Actualizar receta	▼
GET	/api/recipes	Listar recetas	▼
POST	/api/recipes	Crear receta	▼
PATCH	/api/recipes/{id}/deactivate	Desactivar receta	▼
PATCH	/api/recipes/{id}/activate	Activar receta	▼
GET	/api/recipes/search	Buscar recetas	▼
yogurt-batch-controller			^
GET	/api/batches		▼
POST	/api/batches		▼
POST	/api/batches/{batchId}/temperature		▼
POST	/api/batches/{batchId}/refrigeration		▼
POST	/api/batches/{batchId}/inoculating		▼
POST	/api/batches/{batchId}/incubation		▼
POST	/api/batches/{batchId}/heating		▼
POST	/api/batches/{batchId}/fail		▼
POST	/api/batches/{batchId}/complete		▼

Paso 2: Crear una receta (POST)

- Ir al endpoint /api/recipes
- Hacer clic en "Try it out"
- Ingresar el JSON de la receta
- Ejecutar la petición

POST

/api/recipes

Crear receta

^

Permite registrar una nueva receta de yogurt

Parameters

Try it out

No parameters

Request body

required

application/json

▼

Example Value

Schema

```
{
  "name": "Yogurt de fresa",
  "description": "Receta de yogurt natural con frutas",
  "defaultMilkVolume": 1,
  "defaultStarterAmount": 0.5,
  "heatingTemperature": 85,
  "heatingDuration": 30,
  "inoculationTemperature": 45,
  "incubationTemperature": 42,
  "minIncubationTime": 6,
  "maxIncubationTime": 12,
  "refrigerationTime": 4,
  "difficulty": "MEDIUM",
  "tips": "Mantener temperatura constante durante la incubación",
  "ingredients": [
    {
      "name": "string",
      "quantity": 0,
      "unit": "string",
      "notes": "string",
      "optional": true
    }
  ]
}
```

Responses

Code

Description

Links

Paso 3: Consultar datos (GET)

- Ir al endpoint `/api/yogurt-batches`
- Ejecutar la petición GET
- Visualizar los datos obtenidos

GET
/api/recipes/{id}
Obtener receta por ID
^

Retorna una receta específica según su identificador

Parameters
Try it out

Name	Description
id * required integer(\$int64) (path)	id

Responses

Code	Description	Links
200	Receta encontrada	No links

Paso 4: Ver comportamiento del sistema

El sistema permite:

- Creación de lotes de yogurt
- Seguimiento de estados del proceso
- Registro de temperatura durante la producción

4. Base de datos (H2)

La base de datos se ejecuta en memoria durante la ejecución del proyecto.

Acceso local (solo desarrollo):
<http://localhost:8080/h2-console>

5. Evidencias del proyecto

Las evidencias del funcionamiento del sistema se encuentran en la carpeta:

docs/evidencias/

Incluyen:

- Swagger funcionando
- API desplegada en Render
- Pruebas de endpoints GET y POST
- Capturas del sistema en ejecución

6. Arquitectura del sistema

El proyecto está organizado en capas:

- Controller: manejo de endpoints REST
- Service: lógica de negocio

- Repository: acceso a datos
 - Model: entidades del sistema
 - DTO: transferencia de datos
 - Exception Handler: manejo global de errores
-

7. Autor

Cristian Danilo Quintero Montoya
Estudiante de Desarrollo de Software